

Administration Serveur Base de Données

Postgres

Sécurité de Postgres

« une composante essentielle »

La sécurité d'un SGBD

- ☐ Se protéger des accès non autorisés ou mal intentionnés
- ☐ Eviter le vol ou la divulgation de données
- ☐ Eviter la corruption de données
- ☐ Limiter les possibilités d'action en fonction des types d'utilisateurs en interdisant les actions « dangereuses » non indispensables.

Les 4 piliers de la sécurité

- ☐ Sécurité réseau
- ☐ Sécurité du transport
- ☐ Sécurité de l'instance du serveur
- ☐ Sécurité au niveau de la base de données

Sécurité réseau

- ❑ **Sockets Unix Domain (UDS)** : accès local restreint via permissions Unix
<https://www.malekal.com/unix-socket-fonctionnement-et-utilisations/>
- ❑ **Socket TCP/IP** : accès distant à travers le réseau
 - ❑ Sélection interface réseau d'écoute
 - ❑ Définition du port d'écoute
- ❑ **Pare-feu** : contrôle du trafic selon ports, adresses et protocoles

Sécurité du transport

- ❑ **TSL/SSL : chiffrement de la communication**
- ❑ **Authentification par certificat + passphrase**
- ❑ **Configuration pour utilisation de SSL :**
 - ❑ `ssl_ciphers`
 - ❑ `ssl_min_protocol_version`
 - ❑ `etc.`

Sécurité de l'instance du serveur

- ☐ Configuration des connexions acceptées :
 - ☐ Origine
 - ☐ Protocole
 - ☐ Base de données autorisées
 - ☐ Utilisateur
 - ☐ Adresse d'origine
 - ☐ Méthode d'authentification à utiliser

Voir le fichier `pg_hba.conf`

Sécurité au niveau de la base de données

- ❑ **Authentication nécessaire à tout utilisateur :**
 - ❑ Utilisation d'un **compte utilisateur**
 - ❑ Privilèges définis par **rôle**
 - ❑ Affectation de **rôles par utilisateur**

Compte utilisateur ⁽¹⁾

☐ Tout est « rôle » :

- ☐ Un compte utilisateur est un rôle avec le droit de se connecter : **LOGIN**
- ☐ Un compte utilisateur comporte un **nom d'utilisateur**, un **mot de passe**, un ensemble de privilèges

☐ Rôles :

- ☐ Un rôle hérite de droits systèmes qui lui sont oui ou non octroyés :

Droits Systèmes : **SUPERUSER, CREATEDB, CREATEROLE, REPLICATION**

☐ Authentification nécessaire

Pour utiliser un serveur BDD, il est nécessaire de se connecter avec un compte d'utilisateur

Un compte utilisateur comporte un nom d'utilisateur, un mot de passe, un ensemble de privilèges

La gestion des rôles ⁽¹⁾

❑ Création d'un compte utilisateur :

```
CREATE ROLE bob LOGIN  
    ENCRYPTED PASSWORD 'md56a4d668735f5517a5355a979bb13a29b'  
    NOSUPERUSER NOINHERIT NOCREATEDB NOCREATEROLE NOREPLICATION;
```

Peu de privilèges mais l'autorisation de se connecter

❑ Le compte postgres (admin général du serveur)

```
CREATE ROLE postgres WITH  
    LOGIN  
    SUPERUSER  
    INHERIT  
    CREATEDB  
    CREATEROLE  
    REPLICATION  
    ENCRYPTED PASSWORD 'md5244af1e2823d5eaeefc42c5096d8260';
```

La gestion des rôles (2)

❑ Héritage de privilèges à partir d'autres rôles :

```
CREATE ROLE pg_monitor WITH  
    NOLOGIN  
    NOSUPERUSER  
    INHERIT  
    NOCREATEDB  
    NOCREATEROLE  
    NOREPLICATION;
```

```
GRANT pg_read_all_settings, pg_read_all_stats, pg_stat_scan_tables TO pg_monitor;
```

❑ Création d'un rôle

```
CREATE ROLE comptable  
    NOSUPERUSER NOINHERIT NOCREATEDB NOCREATEROLE NOREPLICATION;
```

❑ Affectation du Rôle comptable à l'utilisateur Bob

```
GRANT comptable TO bob;
```

Gestion des Droits ⁽¹⁾

□ Définir l'administrateur d'une Bdd :

```
CREATE DATABASE advcomp
WITH
  OWNER = admin
  ENCODING = 'UTF8'
  LC_COLLATE = 'French_France.1252'
  LC_CTYPE = 'French_France.1252'
  TABLESPACE = pg_default
  CONNECTION LIMIT = -1
  IS_TEMPLATE = False;
```

Le rôle défini comme OWNER a tous les droits sur la Bdd

Pour changer de owner :

```
ALTER DATABASE advcomp OWNER TO bob
```

Gestion des Droits ⁽²⁾

☐ Droit sur une Bdd :

- ☐ Le droit de se connecter à une Bdd peut être attribué à un utilisateur

GRANT CONNECT ON DATABASE advcomp TO bob;

- ☐ Tout utilisateur autorisé à se connecter à une Bdd a par défaut tous les droits sur le schéma « public » de cette Bdd :

GRANT ALL ON SCHEMA public TO PUBLIC; (executé à la création du schema “public”)

- ☐ Pour retirer tous les droits sur une Bdd sauf au owner :

REVOKE ALL ON DATABASE advcomp FROM PUBLIC

Gestion des Droits ⁽³⁾

☐ Droits sur les objets d'une Bdd :

- ☐ Tout objet d'un schéma a un utilisateur créateur qui en est propriétaire et qui a tous les droits
- ☐ Les autres utilisateurs n'ont par défaut aucun droit sur l'objet
- ☐ Droits affectables: **SELECT, UPDATE, TRUNCATE, DELETE, REFERENCES, TRIGGER**

Gestion des Droits : Grant

```
GRANT { { SELECT | INSERT | UPDATE | DELETE | RULE | REFERENCES | TRIGGER }  
        [, ...] | ALL [ PRIVILEGES ] }  
ON [ TABLE ] tablename [, ...]  
TO { username | GROUP groupname | PUBLIC } [, ...]  
  
GRANT { { CREATE | TEMPORARY | TEMP } [, ...] | ALL [ PRIVILEGES ] }  
ON DATABASE dbname [, ...]  
TO { username | GROUP groupname | PUBLIC } [, ...]
```

<https://www.postgresql.org/docs/7.3/sql-grant.html>

Gestion des Droits : Grant

```
GRANT { EXECUTE | ALL [ PRIVILEGES ] }  
    ON FUNCTION funcname ([type, ...]) [, ...]  
    TO { username | GROUP groupname | PUBLIC } [, ...]
```

```
GRANT { USAGE | ALL [ PRIVILEGES ] }  
    ON LANGUAGE langname [, ...]  
    TO { username | GROUP groupname | PUBLIC } [, ...]
```

```
GRANT { { CREATE | USAGE } [, ...] | ALL [ PRIVILEGES ] }  
    ON SCHEMA schemaname [, ...]  
    TO { username | GROUP groupname | PUBLIC } [, ...]
```

<https://www.postgresql.org/docs/7.3/sql-grant.html>

Gestion des Droits : Revoke

```
REVOKE { { SELECT | INSERT | UPDATE | DELETE | RULE | REFERENCES | TRIGGER }  
        [, ...] | ALL [ PRIVILEGES ] }  
ON [ TABLE ] tablename [, ...]  
FROM { username | GROUP groupname | PUBLIC } [, ...]
```

```
REVOKE { { CREATE | TEMPORARY | TEMP } [, ...] | ALL [ PRIVILEGES ] }  
ON DATABASE dbname [, ...]  
FROM { username | GROUP groupname | PUBLIC } [, ...]
```

<https://www.postgresql.org/docs/7.3/sql-revoke.html>

Gestion des Droits : Revoke

```
REVOKE { EXECUTE | ALL [ PRIVILEGES ] }  
  ON FUNCTION funcname ([type, ...]) [, ...]  
  FROM { username | GROUP groupname | PUBLIC } [, ...]
```

```
REVOKE { USAGE | ALL [ PRIVILEGES ] }  
  ON LANGUAGE langname [, ...]  
  FROM { username | GROUP groupname | PUBLIC } [, ...]
```

```
REVOKE { { CREATE | USAGE } [, ...] | ALL [ PRIVILEGES ] }  
  ON SCHEMA schemaname [, ...]  
  FROM { username | GROUP groupname | PUBLIC } [, ...]
```

<https://www.postgresql.org/docs/7.3/sql-revoke.html>

Transaction

- ❑ Un Informatique , une transaction est **une opération cohérente composée d'une séquence de tâches unitaires**.
- ❑ Une transaction sur les données est un **ensemble d'actions** demandées par un seul utilisateur ou un programme, **qui lit et modifie le contenu d'une base de données**.
- ❑ C'est une **unité logique de travail sur les données**
- ❑ Une transaction est **démarquée par un début et une fin :**
Déclaration début, opérations unitaires ..., Fin avec Validation ou Invalidation
- ❑ **A l'issu d'une transaction la base de données doit être dans un état cohérent**
(cohérence des données)

Un exemple de Transaction

❑ Exemple de transaction :

Transfert entre 2 comptes bancaires dans la base de données:

« Transfert de 100€ d'un Compte1 vers un Compte2 »

(1) Débiter Compte1 de 100€ (solde $\leftarrow$ solde - 100)

(2) Créditer Compte2 de 100€ (solde $\leftarrow$ solde + 100)

❑ Après le transfert pour que la BDD soit cohérente :

Tout montant débité d'un compte doit avoir été crédité sur un autre

*« Que se passe-t-il si l'opération de transfert est interrompue après **(1)** ? »*

Donc : (1) et (2) doivent avoir été effectuées avec succès ou non effectuées

(La première propriété d'une transaction est l'**Atomicité**)

Cycle de vie d'une Transaction

- ❑ **Début** : avant toute opération de modification une Transaction doit être démarrée par la primitive **Begin**
- ❑ **Vie de la transaction** : les opérations de lectures et modifications sont réalisées séquentiellement.
- ❑ **Echec** : Une opération a échouée. La transaction doit être invalidée et toutes les modifications effectuées doivent être annulées. La demande d'invalidation va être effectuée par la primitive **Rollback**
- ❑ **Fin avec succès** : Toutes les opérations ont été réalisées sans aucun échec. Les modifications doivent devenir :
 - Durables et exister dans la Bdd
 - Visibles aux autres transactionsLa fin de la transaction avec validation est notifiée par la primitive **Commit**

Propriétés des transactions: ACID

- ❑ **Atomicité** : Les opérations élémentaires de modification des données d'une transaction ont toutes été effectuées ou bien aucune. (opération Atomique : Succès ou Echec)
- ❑ **Cohérence** : Une transaction transforme la base de donnée en partant d'un état cohérent vers un nouvel état cohérent lui aussi.
- ❑ **Isolation** : Les transactions doivent s'exécuter de manière indépendante les unes des autres.
- ❑ **Durabilité**: Les modifications réalisées par une transaction achevée correctement sont inscrites de manière durable.

Les Difficultés

- ❑ **Difficulté de Garantir l'Atomicité :**

Pouvoir « **Défaire** » / « **Annuler** » les mises à jours réalisées si une opération échoue avant la validation de la transaction.

- ❑ **Difficulté de garantir la Cohérence et L'isolation :**

Plusieurs opérations simultanées sur une Bdd.

*Comment garantir la «**Cohérence** » / «**Validité** » des données manipulées ?*

Il faut gérer les accès concurrents

Implémentation de solutions ⁽¹⁾

☐ Atomicité :

☐ Historisation des opérations dans un Journal :

Ecriture de l'opération dans un fichier historique avant son exécution

- Possibilité de **défaire** (réalisation opération inverse)

- Possibilité de **refaire en cas de crash** (rejouer les opérations depuis une date)

☐ Autre solution: historisation Image avant, Image après

Implémentation de solutions (2)

❑ Problème de la concurrence :

Plusieurs transactions s'exécutent en parallèle (concurrence) :

L'accès à une même données dans l'intention de la **modifier** doit être **évité**

« Risque de débiter un compte déjà vide »

Solution : la mise en place de verrouillages au niveau Table ou Ligne

- **Verrou pessimiste** : interdiction aux opérations simultanées de lecture et écriture sur une table, page ou enregistrement. La transaction qui veut acquérir une donnée verrouillée est bloquée jusqu'à sa libération

(Risque d'interblocage)

- **Verrou optimiste** : estampille indicative du numéro de version d'une donnée. Toute modification conduit à une incrémentation de l'estampille. Si la version a changé entre la lecture et la mise à jour: la données n'est plus cohérente et sa mise à jour est interdite.

Implémentation de solutions (3)

- ❑ **Verrouillage pessimiste** : le SGBD met en attente les accès aux données verrouillées (données modifiées par une autre transaction mais non encore « Commitées »)

Avantages : On ne modifie que ce qui peut l'être, pas de modification à annuler

Inconvénients : risque d'interblocage, risque de timeout

- ❑ **Optimiste** : le SGBD laisse accéder à toutes les données. Au moment d'enregistrer une donnée modifiée, le SGBD vérifie que cette donnée n'a pas été modifiée par une autre transaction (Modif → ERREUR).

Avantages : Pas d'interblocage ni de risque de timeout

Inconvénients : Si la BDD est utilisée par une application interactive comment faire en cas de refus de mise à jour ?

Problème du Dirty Read

- ❑ Une transaction **T1** lit des données **créées par une autre transaction en cours T2**
- ❑ **Que se passe-t-il en cas de rollback de T2 ?**

T1 manipule et base ses décisions sur des données incorrectes...

Il faudrait interdire la lecture des données d'une transaction non validée.

Comment ? En verrouillant les données modifiées jusqu'à leur validation.

Problème du Non-Repeatable Read

- ❑ Une transaction **T1** lit une donnée **D1**
- ❑ Durant la transaction **T1** des opérations sont réalisées **sans modifier D1**
- ❑ La transaction **T1** relit la donnée **D1** mais sa valeur est différente :
Elle a été modifiée par une autre transaction !!!

Il faudrait interdire à une Transaction, la modification des données en cours d'utilisation par une autre Transaction.

Comment ? En verrouillant certaines données lues jusqu'à la fin de la transaction.

Problème du Phantom Read

- ❑ Un calcul effectué plusieurs fois dans une transaction **T1** donne des résultats différents

Une transaction T2 a créé de nouvelles données utilisées dans le calcul

Il faudrait pouvoir interdire la lecture des données créées par d'autres transactions après le début de la transaction courante.

Comment ? En excluant les données créées depuis le début de la transaction.

Niveau d'isolation

- ❑ **Niveau d'isolation pour éviter ces problèmes ou les limiter :**
 - **Read uncommitted** : permet les Dirty Read (peu d'isolement, Pas ACID)
 - **Read committed** : évite les Dirty Read mais pas les non-repeatable reads
 - **Repeatable Read** : évite les non-repeatable reads. Ne permet pas d'éviter les Phantom.
 - **Serializable** : isolement parfait, les transactions s'effectuent de manière séquentielle (énormément de verrouillages).

Transaction avec PostgreSQL (1)

- ❑ Un mouvement d'un compte à l'autre :

```
UPDATE Compte SET solde = solde - 100 WHERE numero = 'CPT0001';
```

```
UPDATE Compte SET solde = solde + 100 WHERE numero = 'CPT0002';
```

Si le compte 'CPT0001' est débité et qu'une erreur se produit au moment de créditer 'CPT0002'

!!! La comptabilité n'est plus équilibrée !!!

- ❑ Il faut que **les 2 opérations** de mise à jour se produisent **ou bien Aucune**
- ❑ Les SGBD mettent à disposition un mécanisme pour résoudre ce type de problème:
« La Transaction »
- ❑ Une Transaction est un ensemble de modifications délimité par un début et une fin :

```
BEGIN; -- début de la transaction
```

```
    UPDATE Compte SET solde = solde - 100 WHERE numero = 'CPT0001';
```

```
    UPDATE Compte SET solde = solde + 100 WHERE numero = 'CPT0002';
```

```
COMMIT; -- fin de la transaction|
```

Transaction avec PostgreSQL (2)

☐ Déroulement d'une Transaction:

- ☐ Le mot-clé « **BEGIN** » permet de démarrer la Transaction (création d'un point de retour)
 - ☐ Le mot-clé « **COMMIT** » termine la transaction en validant l'ensemble des modifications effectuées
 - ☐ Si une erreur se produit le SGBD annule toutes les modifications effectuées et la BDD retrouve l'état dans lequel elle était au moment du « **BEGIN** »
 - ☐ Il est possible de forcer l'annulation des modifications en utilisant le mot-clé « **ROLLBACK** »
-
- ☐ En l'absence de déclaration explicite de transaction avec « **BEGIN** », chaque commande SQL est exécutée dans une transaction implicite.
 - ☐ les modification effectuées dans une Transaction ne sont pas visibles aux autres jusqu'au « **COMMIT** ». (Verrouillage pour assurer la cohérence de la BDD)

Les Procédures Stockées (1)

- ❑ Séquence d'instructions qui sont stockées dans la BDD
 - ❑ **Programmation impérative:** SQL, Variables, Itérations, Conditions, Appel d'autres procédures
Avantages: principe de RPC traitement réalisé au plus près des données.
En Postgres plusieurs **langages pour écrire les procédures:** SQL, PL/SQL, C
- ❑ **Procédures écrites en SQL :** liste de requêtes exécutées séquentiellement

```
-- Suppression de la procédure
DROP FUNCTION IF EXISTS mouvement(character varying, character varying, double precision);

-- Création de la procédure
CREATE FUNCTION mouvement(IN compte_origine character varying,
                           IN compte_destination character varying,
                           IN montant double precision)
RETURNS double precision
AS
'
UPDATE compte SET solde = solde - $3 WHERE numero = compte_origine;
UPDATE compte SET solde = solde + $3 WHERE numero = compte_destination;
SELECT solde FROM compte WHERE numero = compte_destination;
'
LANGUAGE SQL;
```

Les Procédures Stockées ⁽²⁾

- ❑ **Procédures écrites en PL/SQL** : langage dédié à l'écriture de procédures
- ❑ Langage utilisé par **Oracle** et **Postgres** :
 - ❑ **Variables** : variables locales, variables d'environnement
 - ❑ **Structures de contrôle** : Conditions, Itérations
 - ❑ **Accès aux données** : Exécution SQL, Curseurs pour lecture ou modification
 - ❑ **Gestion des Exceptions** : Anomalies procédure ou Système

Les Procédures Stockées (3)

❏ Procédure écrite en PL/SQL :

```
CREATE OR REPLACE FUNCTION mouvement_plsql(IN compte_origine character varying,
                                           IN compte_destination character varying,
                                           IN montant double precision)

RETURNS double precision AS
$$|
DECLARE
    solde_origine double precision;
    solde_apres double precision;
    errmsg text;

BEGIN
    -- contrôle disponibilité
    SELECT solde INTO solde_origine FROM compte WHERE numero = compte_origine;
    IF (solde_origine < montant) THEN
        RAISE EXCEPTION 'PAS_DE_CREDIT solde=%', solde_origine;
    END IF;

    UPDATE compte SET solde = solde - montant WHERE numero = compte_origine;
    UPDATE compte SET solde = solde + montant WHERE numero = compte_destination;
    -- init variable
    SELECT solde INTO solde_apres FROM compte WHERE numero = compte_destination;
    RETURN solde_apres;

END;
$$
LANGUAGE 'plpgsql';
```

Les Déclencheurs (Triggers) ⁽¹⁾

- ❑ Procédure stockée déclenchée sur un évènement :
INSERT, DELETE ou **UPDATE** se produisant sur une table.
- ❑ La procédure est déclenchée **Avant, Après** ou **à la place** de l'évènement:
BEFORE, AFTER ou **INSTEAD**.

Exemple:

- On veut tracer de manière automatique les mouvements entre comptes.
- On va placer un déclencheur sur toute mise à jour de la table compte.
- Ce déclencheur va inscrire une ligne trace dans une table « mouvement »

Les Déclencheurs (Triggers) (2)

```
-- Suppression de la procédure de trace de mouvement
```

```
DROP FUNCTION IF EXISTS trace_mouvement() CASCADE;
```

```
-- Création de la procédure trace de mouvement
```

```
CREATE FUNCTION trace_mouvement() RETURNS trigger AS
```

```
$$
```

```
BEGIN
```

```
    INSERT INTO mouvement VALUES (CURRENT_TIMESTAMP, old.numero,
```

```
                                   CASE WHEN new.solde - old.solde > 0 THEN 'Crédit '
                                   ELSE 'Débit '
                                   END,
```

```
                                   END,
```

```
                                   new.solde - old.solde);
```

```
    RETURN NEW;
```

```
END;
```

```
$$
```

```
LANGUAGE 'plpgsql';
```

```
-- Déclaration du trigger
```

```
CREATE TRIGGER trace_mouvement_trigger
```

```
AFTER UPDATE ON compte FOR EACH ROW |
```

```
    EXECUTE PROCEDURE trace_mouvement();
```

```
END;
```

```
-- Création table pour tracer les mouvements
```

```
DROP TABLE IF EXISTS mouvement;
```

```
CREATE TABLE mouvement
```

```
(
```

```
    date timestamp with time zone,
```

```
    compte character varying(10),
```

```
    libelle character varying(50),
```

```
    montant double precision
```

```
);
```

Le planificateur de requêtes ⁽¹⁾

- ❑ Chargé d'optimiser la vitesse d'exécution des requêtes.
- ❑ Il sélectionne le plan d'exécution le plus favorable (théoriquement)
- ❑ **Plan d'exécution** : tables, types d'opérations, ordre de ces opérations
- ❑ Les critères pour choisir le plan d'exécution :
 - ❑ **Les statistiques** : nombre de lignes des tables, nombre de valeurs différentes des colonnes (mise à jour régulière des statistiques nécessaire)
 - ❑ **Les index existants**: pour optimiser les jointures, les restrictions, les tris

Le planificateur de requêtes de Postgres⁽²⁾

- L'instruction **EXPLAIN** de Postgres permet d'obtenir le plan sélectionné par planificateur de requête :

Demande du plan pour la liste des comptes ordonnés par numéro:

```
EXPLAIN SELECT * FROM compte ORDER BY numero DESC
```

Le plan est retourné sous forme textuelle: descriptions des opérations (de dernière à première):

```
Sort (cost=24.76..25.49 rows=290 width=250)
  Sort Key: numero
  -> Seq Scan on compte (cost=0.00..12.90 rows=290 width=250)
```

Dans l'éditeur de requête de PgAdmin  permet d'affiche le plan sous forme graphique:



Le planificateur de requêtes de Postgres (3)

Requête reconstitution de tous les vols : 5 Tables et 4 Jointures

```
SELECT affectation.vol, affectation.pilote, affectation.avion, affectation.date_vol, affectation.nbpas,
pilote.nom AS nompilote, pilote.adresse, pilote.salaire, pilote.comm, pilote.embauche,
vol.vildep, vol.vilar, vol.dep, vol.ar, vol.ch_jour,
avion.nuavion, avion.typeappareil, avion.annserv, avion.nom AS nomavion, avion.nbhvol,
appareil.nbplace, appareil.design
FROM pilote
JOIN affectation ON pilote.nopilote = affectation.pilote
JOIN vol ON affectation.vol = vol.novol
JOIN avion ON affectation.avion = avion.nuavion
JOIN appareil ON avion.typeappareil = appareil.codetype;
```

Coût estimé de 7.87 à 13.68 (vue partielle):

Hash Join (cost=7.87..13.68 rows=55 width=262)
Hash Cond: (affectation.pilote = pilote.nopilote)
-> Hash Join (cost=6.42..11.48 rows=55 width=214)
Hash Cond: (affectation.avion = avion.nuavion)
-> Hash Join (cost=3.24..7.54 rows=55 width=65)
Hash Cond: (vol.novol = affectation.vol)

Le planificateur de requêtes de Postgres (4)

Plan textuel complet:

```
Hash Join (cost=7.87..13.68 rows=55 width=262)
  Hash Cond: (affectation.pilote = pilote.nopilote)
    -> Hash Join (cost=6.42..11.48 rows=55 width=214)
      Hash Cond: (affectation.avion = avion.nuavion)
        -> Hash Join (cost=3.24..7.54 rows=55 width=65)
          Hash Cond: (vol.novol = affectation.vol)
            -> Seq Scan on vol (cost=0.00..3.55 rows=55 width=47)
            -> Hash (cost=2.55..2.55 rows=55 width=25)
              -> Seq Scan on affectation (cost=0.00..2.55 rows=55 width=25)
        -> Hash (cost=2.80..2.80 rows=30 width=149)
          -> Hash Join (cost=1.09..2.80 rows=30 width=149)
            Hash Cond: (avion.typeappareil = appareil.codetype)
              -> Seq Scan on avion (cost=0.00..1.30 rows=30 width=29)
              -> Hash (cost=1.04..1.04 rows=4 width=148)
                -> Seq Scan on appareil (cost=0.00..1.04 rows=4 width=148)
          -> Hash (cost=1.20..1.20 rows=20 width=55)
            -> Seq Scan on pilote (cost=0.00..1.20 rows=20 width=55)
```

Le planificateur de requêtes de Postgres (4)

Représentation graphique du plan complet:

- *Aucun index ni clé primaire utilisé.*
- *Lorsque l'on ajoute une restriction sur une clé:*
- *les index et clés primaires sont utilisés uniquement lorsque la valeur n'existe pas dans les statistiques (?)*

